INFOSYS SPRING BOARD INTERNSHIP

TEAM-D

MIRIYALA GEETHIKA

MENTOR : SANGEETHA GOPALAKRISHNAN

DOCUMENT

DAY-1

Introduction and explanation of the Project.

DAY-2

Teams Separation

Creating repository in github.(Readme file, LICENSE)

DAY-3

What is git/github?

PUSH A FILE TO GITHUB COMMANDS

git add . : Adds **ALL modified/new files** to the staging area.

git add app.py

git commit -m "message" : Saves staged changes permanently with a message

git push : Uploads local commits to **remote repository**

git pull

git branch –all

git fetch –all

git stash

git checkout -b newbranchname : Creates a **new branch**

Switches to it immediately

git branch

Python virtual Environment :

python -m venv venv : Creates a **virtual environment** named venv.

python -m venv shakthi

shakthi/Scripts/activate : **Activates** the virtual environment named shakthi.

DAY-4:

SDLC – software Development Lifecycle

SDLC is a structured process used to **plan, build, test, deploy, and maintain software** efficiently and with good quality. Think of it as a roadmap from idea → working software → long-term support.

---

1. Requirement Analysis

- Understand **what the user needs**
- Gather functional & non-functional requirements

---

2. System Design

- Decide **how the system will work**
- Architecture, database design, UI flow, tech stack

---

3. Implementation (Coding)

- Developers write code based on design
- Use programming languages, frameworks, libraries
  Example: Java + Spring Boot backend, React frontend.

---

4. Testing

- Check if the software works correctly
- Find and fix bugs
- Types: Unit, Integration, System, Acceptance testing

---

5. Deployment

- Release software to users or production environment
- Can be manual or automated (CI/CD)

---

6. Maintenance

- Fix bugs, improve performance

Common SDLC Models

- **Waterfall Model** – Linear, simple, rigid
- **Agile Model** – Iterative, flexible, fast feedback
- **Spiral Model** – Risk-driven, suitable for large projects
- **V-Model** – Testing planned alongside development
- **Iterative Model** – Build and improve in cycles

Why SDLC is Important

- Clear structure & planning
- Better quality software
- Reduced cost and risks

SDLC (Software Development Life Cycle):

It is a step-by-step process used to design, develop, test, and maintain software in a structured way.

It helps developers build software in an organized and efficient manner.

Python Documentation with Examples: Hashing, Strings, Dictionary & Sets

1. Hashing & Password Security (bcrypt)

Hashing converts passwords into unreadable format for secure storage.

Install:

```
pip install bcrypt
```

Example Code:

```
import bcrypt

password = "mypassword"

password_bytes = password.encode('utf-8')

salt = bcrypt.gensalt()

hashed = bcrypt.hashpw(password_bytes, salt)

print(hashed)
```

Check Password:

```
bcrypt.checkpw("mypassword".encode('utf-8'), hashed)
```

2. String Methods with Examples

s = " Hello123 "

strip():

```
print(s.strip())
```

replace():

```
print(s.replace("H", "h"))
```

startswith():

```
print(s.startswith("H"))
```

endswith():

```
print(s.endswith("3"))
```

find():

```
print(s.find("l"))
```

isdigit():

```
print("123".isdigit())
```

isalpha():

```
print("Hello".isalpha())
```

3. Dictionary Methods with Examples

```
d = {"name": "Rashi", "age": 20}
```

get():

```
print(d.get("name"))
```

keys():

```
print(d.keys())
```

values():

```
print(d.values())
```

items():

```
print(d.items())
```

update():

```
d.update({"city": "Indore"})
print(d)
```

pop():

```
d.pop("age")
print(d)
```

clear():

```
d.clear()
print(d)
```

4. Set Methods & Operations with Examples

```
a = {1,2,3}
b = {3,4,5}
```

add():

```
a.add(6)
```

remove():

```
a.remove(2)
```

discard():

```
a.discard(10)
```

union():

```
print(a.union(b))
```

intersection():

```
print(a.intersection(b))
```

difference():

```
print(a.difference(b))
```

Flask API & FastAPI – Python Web Frameworks:

Both Flask and FastAPI are used to build backend APIs in Python. They help your frontend (HTML/JS/React) talk to your server.

Flask API:

Flask is a lightweight web framework used to build simple and flexible APIs.

#program:

```
from flask import Flask, jsonify, request
app = Flask(__name__)
@app.route("/")
def home():
    return "Welcome to Flask API"
@app.route("/user", methods=["GET"])
def get_user():
    return jsonify({"name": "Rashi", "role": "Student"})
@app.route("/login", methods=["POST"])
def login():
    data = request.json
    return jsonify({"message": "Login successful", "user": data})
if __name__ == "__main__":
    app.run(debug=True)
```

FastAPI:

FastAPI is a modern, fast, and high-performance framework for building APIs with automatic docs (Swagger UI).

#program :

```
from fastapi import FastAPI
from pydantic import BaseModel
app = FastAPI()
class User(BaseModel):
    name: str
```

```
    email: str
@app.get("/")
def home():
    return {"message": "Welcome to FastAPI"}
@app.get("/user")
def get_user():
    return {"name": "Rashi", "role": "Student"}
@app.post("/register")
def register(user: User):
    return {"message": "Registered successfully", "user": user}
```

JWT (JSON Web Token):

JWT stands for JSON Web Token.
It is a secure way to send user identity data between client and server.

➞ Used in Login systems
➞ Used in Flask / FastAPI / Node APIs
➞ Works without server-side sessions (stateless)

#Basic git commands

Check Git Version

```
git --version
```

Shows which version of Git is installed.

---

2. Initialize a Repository

git init

Creates a new Git repository in your project folder.

---

3. Check Status

git status

Shows the status of files (modified, staged, untracked).

---

4. Add Files to Staging Area

git add .

Adds all changed files to staging.

Or for a specific file:

git add filename.txt

---

5. Commit Changes

git commit -m "Your commit message"

Saves your changes with a message.

---

6. Check Commit History

git log

Shows commit history.

---

7. Connect to Remote Repository

git remote add origin https://github.com/username/repo-name.git

---

8. Push Code to GitHub

git push -u origin main

---

9. Clone a Repository

git clone https://github.com/username/repo-name.git

---

10. Pull Latest Changes

git pull origin main

---

11. Create a New Branch

git branch new-branch-name

---

12. Switch Branch

git checkout new-branch-name

or

git switch new-branch-name

---

13. Merge Branch

git merge new-branch-name

---

14. Remove File from Git

git rm filename.txt

---

15. Undo Last Commit (Soft Reset)

git reset --soft HEAD~1

Python Functions: #File I/O Methods: ..with one example

Python File I/O Methods

Input/Output

File I/O in Python is used to **read from and write to files**.

1. open()

Used to open a file.

Syntax:

file = open("filename.txt", "mode")

Common Modes:

- "r" → Read
- "w" → Write (overwrites file)
- "a" → Append
- "x" → Create new file
- "b" → Binary mode
- "t" → Text mode (default)

2. read()

Reads entire file content.

file.read()

3. readline()

Reads one line at a time.

file.readline()

4. readlines()

Reads all lines and returns them as a list.

file.readlines()

5. write()

Writes content to a file.

file.write("Hello World")

6. close()

Closes the file.

file.close()

Python Built-in Functions

Built-in functions are **predefined functions** in Python that are always available without importing any module.

1. print()

Displays output on the screen.

print("Hello, World!")

2. input()

Takes user input from keyboard.

```
name = input("Enter your name: ")
print(name)
```

3. type()

Returns the data type of a variable.

```
x = 10
print(type(x))   # <class 'int'>
```

4. len()

Returns the length of an object.

```
text = "Python"
print(len(text))   # 6
```

5. range()

Generates a sequence of numbers.

```
for i in range(1, 5):
    print(i)
```

6. sum()

Returns the sum of elements.

```
numbers = [1, 2, 3]
print(sum(numbers))   # 6
```

7. max() and min()

Returns maximum and minimum values.

```
nums = [5, 2, 9, 1]
print(max(nums))  # 9
print(min(nums))  # 1
```

8. abs()

Returns absolute (positive) value.

```
print(abs(-10))   # 10
```

9. round()

Rounds a number.

```
print(round(3.456, 2))   # 3.46
```

10. enumerate()

Returns index and value while looping.

Python Conversion Functions

Conversion functions are used to **convert one data type into another data type**.

1. int()

Converts a value to an integer.

```
x = "10"
y = int(x)
print(y)       # 10
print(type(y))  # <class 'int'>
```

2. float()

Converts a value to a floating-point number.

```
x = "3.14"
y = float(x)
print(y)      # 3.14
```

3. str()

Converts a value to a string.

```
x = 100
y = str(x)
print(y)      # "100"
print(type(y))  # <class 'str'>
```

4. bool()

Converts a value to Boolean (True/False).

```
print(bool(1))   # True
print(bool(0))   # False
print(bool(""))  # False
```

5. list()

Converts to a list.

```
x = "Python"
print(list(x))
# ['P', 'y', 't', 'h', 'o', 'n']
```

6. tuple()

Converts to a tuple.

```
x = [1, 2, 3]
print(tuple(x))
# (1, 2, 3)
```

7. set()

Converts to a set (removes duplicates).

```
x = [1, 2, 2, 3]
print(set(x))
# {1, 2, 3}
```

8. dict()

Converts to a dictionary (requires key-value pairs).

```
x = [(1, "a"), (2, "b")]
print(dict(x))
# {1: 'a', 2: 'b'}
```

AI MODELS

AI models are computer programs trained on data to recognize patterns, make decisions, or generate content similar to human intelligence.

They learn from data, identify patterns, and then use that knowledge to:

- Open AI
- GPT
- Gemini

🔹 Main Types of AI Models (With Examples)

Machine Learning (ML) Models

Learn patterns from structured data.

Examples:

- Spam email detection
- Credit card fraud detection

Common algorithms:

- Linear Regression
- Decision Trees
- Random Forest

Rate Limit:

The maximum number of API requests you can send within a certain time (per minute/hour).

Token Limit:

The maximum number of tokens (words + characters split into pieces) allowed in input + output.

LLM

An LLM (Large Language Model) is a type of NLP model trained on massive amounts of text data to understand and generate human-like language.

An **LLM**:

- Uses deep learning (usually Transformer architecture)
- Is trained on billions/trillions of words
- Can generate text, answer questions, summarize, translate, and even write code

Examples of LLMs

- OpenAI's GPT models
- Google's Gemini
- Meta's LLaMA

NLP

NLP (Natural Language Processing) is a broader field of AI that focuses on enabling computers to understand and process human language.

It includes:

- Text classification
- Sentiment analysis
- Translation
- Speech recognition
- Chatbots

Natural Language Processing (NLP) Models

Understand and generate human language.

Examples:

- Chatbots
- Language translation
- Text summarization

Popular models:

- OpenAI's GPT models
- Google's BERT

Install open ai using terminal. – import openai

pip install openai

Gemini, Open AI and Groq

API key generation. – open Ai

Google Gemini AI

Llama using groq

Computer Vision Models

Understand images and videos.

Examples:

- Object detection in CCTV
- Medical image diagnosis
- Barcode scanning

Generative AI Models

Create new content (text, images, music).

Examples:

- Text generation (like ChatGPT)
- AI image generation
- AI music creation

Examples:

- OpenAI (DALL·E)
- Stability AI (Stable Diffusion)

NLP (Natural Language Processing)

NLP is a field of AI that helps computers understand, interpret, and generate human language.

Example:

- Chatbots
- Sentiment Analysis
- Machine Translation

NLP Implementation Steps

1.Collecting the Data:

Gather text data from:

- Websites
- CSV files
- Social media

- APIs

Example:

text = "learning NLP and AI from google."

NLTK (Natural Language Toolkit):

NLTK is a Python library used for NLP tasks like tokenization, stemming, tagging, etc.

Install:

pip install nltk

Import:

import nlt

Clean and convert text into numerical format.

Includes:

- Lowercasing
- Removing punctuation
- Removing stopwords
- Tokenization
- Vectorization (Bag of Words, TF-IDF)

Example:

text = text.lower()

NLTK (Natural Language Toolkit):

NLTK is a Python library used for NLP tasks like tokenization, stemming, tagging, etc.

Install:

pip install nltk

Import:

```
import nltk
```

Key Steps in NLTK:

Tokenization: Breaking text into words or sentences.

Example:

```
from nltk.tokenize import word_tokenize
text = "Rashi loves NLP."
tokens = word_tokenize(text)
print(tokens)
```

Output:

['Geethu', 'loves', 'NLP', '.']

Stop Words Removal : Removing common words (is, the, and, etc.)

Example:

```
from nltk.corpus import stopwords
stop_words = set(stopwords.words("english"))
filtered = [word for word in tokens if word.lower() not in stop_words]
print(filtered)
```

Output:

['Geethika, 'loves', 'NLP', '.']

Stemming: Reducing words to root form.

Example:

```
from nltk.stem import PorterStemmer
ps = PorterStemmer()
print(ps.stem("playing"))
```

Output:

play

Lemmatization: Converts word to meaningful base form.

Example:

```
from nltk.stem import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()
print(lemmatizer.lemmatize("running", pos="v"))
```

Output:

run

Difference:

- Stemming → playi (sometimes incorrect)
- Lemmatization → play (correct dictionary form)

POS Tagging (Part of Speech Tagging): Identifies grammar role of words (noun, verb, adjective).

Example:

```
nltk.download('averaged_perceptron_tagger')
words = word_tokenize("Rashi loves NLP")
print(nltk.pos_tag(words))
```

Output:

[('Rashi', 'NNP'), ('loves', 'VBZ'), ('NLP', 'NNP')]

NER (Named Entity Recognition): Identifies names of people, places, organizations.

Example:

```
nltk.download('maxent_ne_chunker')
nltk.download('words')
from nltk import ne_chunk
sentence = nltk.pos_tag(word_tokenize("Rashi lives in India"))
print(ne_chunk(sentence))
```

Output:

```
(S (PERSON Rashi) lives in (GPE India))
```

```
# Required Downloads -
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('averaged_perceptron_tagger')
nltk.download('maxent_ne_chunker')
nltk.download('words')
```

Clustering

- **Type:** Unsupervised Learning
- **Purpose:** Group similar data points together without labeled outputs.
- **Goal:** Maximize similarity within a cluster and minimize similarity between clusters.
- **Examples of Algorithms:**
    - K-Means
    - Hierarchical Clustering
    - DBSCAN

Applications: Customer segmentation, document grouping, image segmentation.

---

2) Embedding

- **Definition:** A technique that converts data (text, images, etc.) into dense numerical vectors.
- **Purpose:** Capture semantic meaning in numerical form.
- **Output:** Low-dimensional vector representation.

Example:

- "King" and "Queen" will have similar embedding vectors.

Used In: NLP, recommendation systems, semantic search.

---

3) Conversion of Text to Number (Vectorization)

- **Definition:** Process of converting text into numerical format so machines can understand it.
- **Common Methods:**
  - Bag of Words (BoW)
  - TF-IDF
  - Word2Vec
  - One-Hot Encoding

Why Needed?
Machine learning models work only with numbers, not raw text.

---

4) Recommended Systems (Recommendation Systems)

- **Definition:** Systems that suggest products/services/content to users.
- **Types:**

1. Content-Based Filtering
2. Collaborative Filtering
3. Hybrid Methods

Examples:

- Amazon product recommendations
- Netflix movie suggestions

---

5) Logistic Regression

- **Type:** Supervised Classification Algorithm
- **Used For:** Binary classification (0 or 1, Yes or No).
- **Function Used:** Sigmoid Function

Sigmoid Formula:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Prediction Rule:

- If **$\hat{y} \geq 0.5$ → Class = 1**
- If **$\hat{y} < 0.5$ → Class = 0**

Applications: Spam detection, disease prediction, fraud detection.

Decision Tree

- **Type:** Supervised Learning Algorithm

- **Used For:** Classification and Regression
- **Structure:** Tree-like model with:
  - Root node (first decision)
  - Internal nodes (conditions)
  - Leaf nodes (final output)
- **Working:**
  Splits data based on feature conditions (e.g., Age < 30?) until a final decision (Fit/Unfit) is reached.
- **Advantages:**
  - Easy to understand
  - Visual representation
  - No need for feature scaling
- **Common Metrics Used:**
  - Gini Index
  - Entropy (Information Gain)
- **Applications:**
  Medical diagnosis, credit approval, fraud detection.

Sklearn (Scikit-learn)

- **Definition:** A popular Python machine learning library.
- **Full Name:** Scikit-learn
- **Used For:**
  - Classification
  - Regression
  - Clustering
  - Model evaluation

- **Key Features:**
    - Simple and consistent API
    - Built on NumPy, Pandas, Matplotlib
    - Provides many ML algorithms

Example Import:

```
import sklearn
```

Logistic Regression in Sklearn

To use Logistic Regression in Python:

```
from sklearn.linear_model import LogisticRegression
```

- **Purpose:** Binary classification (0/1)
- **Uses:** Sigmoid function
- **Output:** Probability between 0 and 1

Basic Steps:

```
model = LogisticRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)
```

Machine Learning (ML)

Machine Learning is a branch of Artificial Intelligence that enables computers to learn from data and make decisions without being explicitly programmed.

What is Machine Learning?

Machine Learning (ML) is a technology that allows computers to learn patterns from data and improve their performance over time.

Example:
If we give many emails labeled as “spam” or “not spam,” the system learns how to identify spam emails automatically.

Types of Machine Learning

1. **Supervised Learning**
    - Learns from labeled data.
    - Example: Predicting house prices.
2. **Unsupervised Learning**
    - Finds patterns in unlabeled data.
    - Example: Grouping customers based on buying behavior.
3. **Reinforcement Learning**
    - Learns through rewards and punishments.
    - Example: Self-driving cars, game-playing AI.

Applications of Machine Learning

- Spam email detection
- Face recognition
- Medical diagnosis
- Recommendation systems (Netflix, Amazon)
- Chatbots (like healthcare assistants)

Advantages

- Improves accuracy over time
- Automates complex tasks

K-Means Algorithm

🔹 **K-Means Clustering**

✅ **Type:**

- **Unsupervised Learning**
- Used for **Clustering**

🔹 **Meaning:**

- **K** → Number of clusters
- **Means** → Average (Centroid of cluster)

🔹 **Steps of K-Means Algorithm**

1. Choose number of clusters → **K = 3** (example)
2. Randomly initialize **centroids**
3. Calculate **Euclidean Distance** between each data point and centroids
4. Assign points to nearest centroid
5. Recalculate centroids (take average of points)
6. Repeat until centroids stop changing (convergence)

🔹 **Euclidean Distance Formula**

$$d = \sqrt{(x2 - x1)^2 + (y2 - y1)^2}$$

🔹 Python Code (K-Means)

```
from sklearn.cluster import KMeans
import numpy as np
import matplotlib.pyplot as plt

# Sample data
X = np.array([[1,2], [2,3], [3,4], [8,7], [9,8], [10,9]])

# Create model
kmeans = KMeans(n_clusters=3, random_state=42)

# Fit model
kmeans.fit(X)

# Get cluster labels
labels = kmeans.labels_

# Get centroids
centroids = kmeans.cluster_centers_

print("Labels:", labels)
print("Centroids:", centroids)
```

🔹 Applications of K-Means

- Customer segmentation
- Market analysis
- Image compression
- Pattern recognition

📅 26/02 – KNN Algorithm

🔹 K-Nearest Neighbors (KNN)

✅ Type:

- **Supervised Learning**
- Used for:
  - **Classification**
  - **Regression**

🔹 Type of Learning:

- **Lazy Learning**
  - No training phase
  - Stores entire dataset
  - Prediction happens during testing

🔹 Working Steps of KNN

1. Choose value of **K**
2. Calculate distance between test point and all training points
3. Find **K nearest points**
4. For Classification → Majority voting
5. For Regression → Average of nearest values

🔹 Distance Metrics

🔹 Euclidean Distance (Most Common)

$$d = \sqrt{(x2 - x1)^2 + (y2 - y1)^2}$$

🔹 Manhattan Distance

$$d = | x1 - x2 | + | y1 - y2 |$$

- **Python Code (KNN Classification)**

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report

# Sample dataset
X = np.array([[1,2],[2,3],[3,4],[6,7],[7,8],[8,9]])
y = np.array([0,0,0,1,1,1])

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# Feature Scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Create model
knn = KNeighborsClassifier(n_neighbors=3)

# Train model
knn.fit(X_train, y_train)

# Prediction
y_pred = knn.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Applications of KNN

- Recommendation systems
- Medical diagnosis
- Image recognition
- Spam detection

Difference Between K-Means and KNN

Feature	K-Means	KNN
Type	Unsupervised	Supervised
Purpose	Clustering	Classification / Regression
Training Phase	Yes	No (Lazy learning)
Output	Groups	Predicted label/value